

Multi-Tenant Website Builder

Architecture Design Conversation · February 2026

This document is a transcript of a design conversation covering the routing architecture for a multi-tenant website builder platform. Topics covered include: Traefik vs custom router approaches, Redis as a routing cache, CQRS read-model projections, Go implementation of the routing service, event-driven cache invalidation, and the final decision to use an in-memory routing table in Go.

YOU

I'm creating an easy-to-use website builder where each customer can have a multi-page website under his own domain. Currently I have a WordPress instance running as a Docker container with a page builder installed on it. I want to setup a Traefik instance with dynamic routing that would route the incoming requests to the appropriate page on the Wordpress instance. I need to somehow manage a routing table that maps domain/path pairs to page IDs or slugs, so that Traefik could route to the appropriate target page based on the domain and uri path of the incoming request. What are my options?

CLAUDE

This is an interesting architecture challenge. Let me walk you through your realistic options.

The Core Problem

Traefik is a reverse proxy, not a URL rewriter. It can route `customer.com` to your WordPress container, but it can't transform `customer.com/about` to `yourdomain.com/?page_id=42` out of the box. You need something to handle that translation layer.

Option 1: Traefik + Middleware (Rewrite at the Proxy Level)

Traefik has a plugin ecosystem and supports middleware like `redirectRegex` and `stripPrefix`, but these are static rules loaded from config files or Docker labels — not dynamically manageable from a database. You'd need to reload Traefik's config whenever a customer adds a page.

How it works: You maintain a flat file or key-value store (like Consul or Redis) as Traefik's dynamic config provider. When a customer updates their site, your backend writes new routing rules to that store and Traefik picks them up without restarting.

Limitation: Traefik's built-in rewriting is fairly limited. Doing domain+path to WordPress query string rewrites requires a custom plugin or a middleware layer in front of WordPress.

Option 2: Nginx/Caddy as a Middle Layer (Most Practical)

Put a lightweight Nginx or Caddy instance between Traefik and WordPress. Traefik handles TLS termination and routes by domain. Nginx handles the path-to-WordPress-page translation.

```
Customer Browser -> Traefik (TLS, domain routing) -> Nginx (path rewriting) -> WordPress
```

Option 3: A Thin Custom Router Service (Most Flexible)

Build a small service that sits between Traefik and WordPress. Traefik routes all traffic to this service, which reads the incoming Host header and path, looks up the routing table in a database, and proxies the request to WordPress with the correct URL.

```
CREATE TABLE routes (  
  domain TEXT,  
  path TEXT,  
  wp_target TEXT,  
  PRIMARY KEY (domain, path)  
);
```

Option 4: WordPress Handles It Natively (Simplest, But Limiting)

WordPress Multisite with custom domain mapping. Traefik just forwards all traffic to WordPress per domain, and WordPress resolves which site/page to serve.

Recommendation

For a proper multi-tenant website builder, **Option 3 (thin custom router)** is the right long-term architecture. If you want to move fast right now, **Option 4 (WordPress Multisite)** gets you running with the least infrastructure overhead.

YOU

I'm thinking about option 3 but handling this within Traefik still seems a reasonable approach. Assuming that my Wordpress instance has pretty permalinks setup properly, Traefik would just have to work with domain and paths but not query strings. What is your opinion?

CLAUDE

You're right, that changes things significantly. With pretty permalinks enabled, WordPress URLs become clean paths like `/about/`, `/contact/`, `/services/our-team/` etc. — no query strings involved.

With clean paths, Traefik becomes much more viable. Its `PathPrefix` routing and `AddPrefix/StripPrefix` middlewares can handle path rewriting without any custom code. You'd use Traefik's dynamic configuration (file provider or a key-value store like Redis/Consul) to maintain per-domain routing rules.

However, there's still a real limitation: **Traefik's dynamic config is rule-based, not lookup-table-based**. Every domain+path combination needs to be an explicit router entry. If a customer has 50 pages, that's 50 rules. At scale — say 1000 customers with 50 pages each — you're managing 50,000 Traefik router entries.

A middle ground worth considering: **one Traefik rule per customer domain**, routing everything for that domain to WordPress with a prefix rewrite. The path structure on WordPress's side mirrors what the customer sees publicly. Traefik only needs one entry per customer (not per page), and WordPress's own routing handles the rest.

YOU

Each customer will have just a few pages. If Traefik is using a key-value store (e.g. Redis), then why would that be a problem to manage even millions of rules? As far as I know Redis is definitely designed for that.

CLAUDE

You're absolutely right, I overcomplicated that concern. Redis is trivially capable of holding millions of routing rules, and lookups remain $O(1)$ regardless of scale. That objection doesn't hold up.

Traefik's Redis provider reads its dynamic configuration from Redis and watches for changes, so adding or removing a route is just a Redis write — no reloads, no restarts, near-instant propagation.

The architecture would look like:

```
Customer Browser -> Traefik (reads rules from Redis) -> WordPress container  
Your Admin Backend -> writes routing rules -> Redis
```

The main things worth validating before committing to this approach are how Traefik handles very large numbers of router entries in terms of memory footprint and rule compilation time when changes arrive, and whether its Redis provider supports the granularity you need.

YOU

How do you mean memory footprint? I thought that Traefik would just on each request send a key (domain-path pair) to Redis and gets back the slug of the target page. Or is it not the way how is it working?

CLAUDE

That's an important clarification — that's **not** how Traefik's Redis provider works.

Traefik loads its entire configuration from Redis into memory at startup and then watches for changes, updating its in-memory routing table when Redis keys change. It does **not** query Redis on a per-request basis. It's a config provider, not a runtime lookup mechanism.

So Traefik compiles all your routing rules into an in-memory structure and uses that to match incoming requests. This is what makes it fast — there's no network round-trip on each request. But it also means all rules live in Traefik's memory, and every time you add or change a rule, Traefik re-processes its configuration.

If you want the per-request Redis lookup behavior you described, that's exactly the thin custom router from Option 3. This is actually very fast in practice — a Redis lookup on a local network is sub-millisecond. You'd have one single Traefik rule that forwards everything to your router service, and all the routing intelligence lives there.

YOU

Ok, then I would move forward option 3 (custom router). In this case I want to know more about Redis. Is it a persistent store? Is it worth it to make it faster with memcached or similar solution?

CLAUDE

Redis persistence: Yes, Redis supports persistence. It has two mechanisms — RDB (periodic snapshots) and AOF (append-only log of every write). You can use either or both. For a routing table that lives in your primary database, Redis is better thought of as a **cache** rather than the source of truth.

On adding Memcached on top of Redis: This doesn't make sense. Memcached is also a network-based key-value store, so it has the same latency profile as Redis — you wouldn't gain anything. Redis strictly outclasses it for your use case.

What would actually make it faster is keeping the routing table in your router service's own process memory — a simple hash map. You populate it from Redis or Postgres at startup and refresh it periodically or via a pub/sub notification when routes change. That eliminates the network round-trip entirely.

For your scale, even direct Postgres lookups per request would likely be fast enough with proper indexing. Redis is a reasonable caching layer, but don't over-engineer it before you have evidence you need it.

YOU

I will have a primary database but it won't have such flat structure that would be needed for routing lookups. It will store entities like **sites**, **domains**, **pages**, **sections** and other content related stuff in a traditional relational database structure. My plan is to use Redis on top of it as a persistent "projection" of the subset of these data, and whenever my database content updates in a way that affects the routing table, then this change would also be propagated to the Redis lookup table. What do you think?

CLAUDE

This is a well-thought-out approach and a solid application of the **CQRS / read model** pattern. Your relational database is the source of truth optimized for writes and business logic, and Redis is a denormalized read model optimized for a specific query pattern.

Synchronous dual-write is simple but risky — if the Redis write fails, your read model is stale.

Event-driven propagation is more robust. When something affecting routing changes, your backend emits an event that a separate worker consumes and applies to Redis. The downside is eventual consistency.

Periodic reconciliation as a safety net is worth adding regardless. A background job that periodically recomputes the Redis projection from Postgres catches any drift caused by missed events.

One thing to be careful about: making Redis persistence reliable enough that a Redis restart doesn't leave your router service blind. AOF persistence with **appendfsync everysec** is a reasonable tradeoff.

Overall this is a clean architecture — the pattern you're describing is exactly the right tool for this problem.

YOU

What do you mean "making Redis persistence reliable enough"? It isn't reliable? How does it work?

CLAUDE

Redis persistence is reliable in the sense that it works correctly, but it has tradeoffs depending on which mode you use.

RDB (snapshotting): Redis periodically writes a snapshot of the entire dataset to disk. If Redis crashes between snapshots, you lose the writes that happened since the last snapshot — potentially several minutes.

AOF (Append-Only File): Every write operation is logged to a file. On restart Redis replays the log to reconstruct the dataset. The **appendfsync** setting controls how aggressively it flushes to disk — **always** means no data loss but slower writes, **everysec** means at most one second of data loss.

What I meant by "reliable enough" is that you should consciously choose your persistence settings rather than rely on defaults. The default Redis configuration is optimized for performance and is somewhat conservative about persistence.

For your use case though, this concern is minimal because your Postgres database is the source of truth. A Redis restart just means you run your reconciliation job to rebuild the projection. The only real risk is the window between the restart and the reconciliation completing, during which your router service can't resolve routes. You can mitigate this by having your router service fall back to Postgres directly when a Redis key is missing.

YOU

If I understand correctly the **appendfsync always** option would be reasonable for my usecase. I don't care about slower writes, because write operations are very rare compared to lookups. And how does Redis work? Does it keep the whole routing table in its memory while running?

CLAUDE

Yes, exactly — **appendfsync always** is a sensible choice given your write pattern. Rare writes, massive read volume — durability on every write costs you nothing meaningful.

And yes, Redis is an **in-memory database**. The entire dataset lives in RAM at all times. This is the fundamental reason it's so fast — reads and writes are just memory operations, with disk I/O happening asynchronously (or synchronously with **always**) purely for durability, not for serving requests.

On startup Redis loads the AOF log into memory by replaying all the write operations, reconstructing the full dataset. From that point on everything is served from RAM.

For your routing table — even with millions of domain+path entries, where each entry is maybe 100-200 bytes — you're looking at a few hundred megabytes at most. Completely trivial for any modern server.

So the full picture at request time is: incoming request hits your custom router, it does a Redis GET (a memory lookup on the Redis server, returned over a local network), gets the slug back in sub-millisecond time, rewrites the path, and proxies to WordPress.

YOU

Understood. And do I have to extend my backup procedures to backup the persistent storage of Redis as well, or it's better to just rely on the reconsiliation at startup?

CLAUDE

Since your Postgres database is the source of truth and Redis is just a derived projection of it, there's no point backing up Redis separately. Everything in Redis can be reconstructed from Postgres at any time, so a Redis backup would give you nothing that your Postgres backup doesn't already cover.

The clean approach is to treat Redis as disposable infrastructure — if you lose it entirely, you just rebuild the projection from Postgres and you're back.

The only thing worth ensuring is that your reconciliation/rebuild process is fast enough that you're comfortable with the downtime window if Redis goes down cold. Given that a routing table is a small and simple dataset, a full rebuild should be nearly instant.

YOU

Reconsiliation would be part of the Redis container (e.g. as a startup script/plugin), or should it be a separated container? How should I imagine that? My concern with the latter is that if I do an appropriate SQL query to the relational database, it would result in a huge dataset with tens of thousands of rows (each row representing a routing rule), and sending those rows one-by-one to Redis could take relatively long time even in local network. What do you think?

CLAUDE

Where the reconciliation lives: It should be a separate concern from the Redis container itself — Redis is a general-purpose data store and shouldn't know anything about your business data. It's better placed as a small script or service in your admin backend, or as a separate container.

On the performance concern: Your instinct is right, but the solution is straightforward — use Redis pipelining. Instead of sending each SET command individually (one network round-trip per row), you batch all the commands into a single pipeline and send them in one shot. Redis executes them sequentially and returns all responses at once. Tens of thousands of rows would load in well under a second this way.

Additionally, you could use Redis's **MULTI/EXEC** transaction together with pipelining to make the entire rebuild atomic — your router service either sees the old table or the new one, never a half-rebuilt state.

So the flow would be: container starts → reconciliation script runs → opens Postgres connection, runs your denormalizing SQL query → gets all rows → opens Redis pipeline → fires all SET commands in one batch → done.

YOU

Ok, I will go forward with this. But I still have a question: in this case I don't need Redis persistency at all, as the routing table will be rebuilt at every restart, right? And one more concern: What happens if only the Redis container gets restarted? In this case the reconsiliation would not be executed. How to overcome this?

CLAUDE

On persistence: Correct. If you're always rebuilding from Postgres on startup, Redis persistence adds no value and you can disable it entirely. This actually simplifies your Redis configuration and removes the overhead of AOF writes.

On the Redis-only restart problem: This is a real gap. A few ways to handle it:

The simplest solution is to run the reconciliation script as part of the Redis container's startup, not your application stack's startup. You can do this with a custom Docker entrypoint that starts Redis and then runs the reconciliation script against it.

The cleaner solution is to make your router service resilient to missing keys. When it gets a Redis miss, instead of failing the request, it falls back to Postgres directly, and optionally re-populates that specific key in Redis.

In practice the first option (reconciliation as part of Redis container startup) combined with the fallback to Postgres in your router service covers the vast majority of scenarios with minimal complexity.

YOU

Ok, I will implement reconsiliation+fallback to Postgres. Could you draw me a diagram about this solution?

CLAUDE

[Architecture diagram was generated and provided as an HTML file showing three flows:

- ① **Request Flow (hot path):** Browser → Traefik → Custom Router → Redis lookup → WordPress. On Redis MISS → Custom Router queries Postgres, backfills key, proxies to WordPress.
- ② **Startup Reconciliation:** Redis container starts → reconciliation script runs → Postgres denormalizing SQL query → all rows written to Redis via pipelined batch.
- ③ **Write Flow:** Admin Backend writes to Postgres (source of truth) → propagates affected key(s) to Redis.]

YOU

In the meantime I was just thinking about completely omitting routing table update on content change. It would just increase the complexity. Custom router's fallback will anyway backfill the new Redis keys on first miss.

CLAUDE

That's a clean simplification and it works perfectly well. The flow becomes:

- **Startup:** reconciliation rebuilds the full Redis projection from Postgres
- **Runtime:** Redis HIT — instant lookup; Redis MISS — Postgres fallback + backfill

The only edge case worth thinking about is **slug changes or page deletions**. If a customer renames a page slug or deletes a page, the old Redis key becomes stale — it will keep returning the wrong target indefinitely since there's no invalidation step.

The simplest mitigation is a **periodic full reconciliation** — say every few minutes — that rewrites the entire Redis projection from Postgres. Combined with the startup reconciliation you already have, this covers all cases without any per-change propagation logic.

So the final setup would be just two things: reconciliation on Redis startup, and a scheduled periodic reconciliation. No write-path complexity at all.

YOU

Or what about introducing a TTL for the records stored in Redis?

CLAUDE

That's an elegant solution. If every Redis key has a TTL — say 5 or 10 minutes — then:

- Stale keys (renamed/deleted pages) expire automatically within that window
- After expiry the next request gets a miss, falls back to Postgres, and either backfills the new correct value or simply doesn't backfill if the page no longer exists
- No periodic reconciliation job needed at all
- Startup reconciliation still makes sense to pre-warm Redis and avoid a cold start stampede to Postgres

This is actually a more standard caching pattern and arguably simpler than any reconciliation job. The TTL acts as your eventual consistency window, and you tune it based on how quickly you want slug changes to propagate.

YOU

In the meantime, a bit different approach came into my mind. We would use just the domains as keys in Redis, and the corresponding value would contain the whole routing table for that specific domain, or a redirection to another domain (e.g. for handling `www.customer.com` -> `customer.com` kind of redirections. The custom router would be responsible for full reconsilation at startups (or when Redis restarts triggered by an event sent by the container entryptoint script). On updates the admin backend would send an invalidation event about the affected domain(s), so that the custom router can update Redis content accordingly. In addition we would still introduce a longer TTL to cover all edge cases when some update was missed for some reason. How about that one?

CLAUDE

This is a nicely considered design. Using domains as keys means far fewer Redis entries — one per domain instead of one per page. The value would be a small JSON blob:

```
{
  "type": "site",
  "routes": {
    "/about": "about-us",
    "/contact": "contact-page"
  }
}
```

or for redirects:

```
{
  "type": "redirect",
  "target": "customer.com"
}
```

The custom router loads the whole domain blob in one Redis GET, then does the path lookup in-process. This is actually faster than per-route Redis lookups since you have one network round-trip per request regardless of how many pages the site has.

Invalidation events from the admin backend are a clean mechanism. The long TTL as a safety net on top of this is sensible.

One thing to think about: the custom router now holds responsibility for both serving requests and reacting to invalidation events, which means it needs an event listener running alongside the request-handling logic.

Overall this is a solid evolution of the design — fewer Redis keys, one round-trip per request, event-driven invalidation with TTL as a fallback.

YOU

And what technology do you propose for the routing service? I guess that a Laravel app would be overkill, especially when it comes to keeping the already loaded domain routing tables in-memory.

CLAUDE

You're right on both counts — Laravel would be overkill, and the in-memory caching aspect is an important consideration that rules out most traditional request/response frameworks where each request is handled in isolation with no shared state between requests.

What you want is a **long-running process** that holds the Redis-fetched domain blobs in an in-process hash map and serves requests against that.

Go is the strongest fit here. It has an excellent built-in HTTP server, handles high concurrency with goroutines natively, and a simple `sync.Map` or a `map` behind a `RWMutex` gives you the in-process cache with safe concurrent access. The binary is small and starts instantly. The standard library covers everything you need — HTTP server, reverse proxy (`httputil.ReverseProxy`), Redis client. No framework needed at all, just a few hundred lines of idiomatic Go.

Node.js is a reasonable second choice, especially if your Admin Backend is already JavaScript/TypeScript. A plain JS `Map` object as the in-process cache works well.

Rust would give you the best raw performance, but the development overhead is significant compared to Go.

Go is the recommendation.

YOU

In the meantime I was thinking: maybe even the Redis is not needed. The router service could keep all the routing table in its memory. It shouldn't be huge amount of data. Or when if it will be, we can introduce a Redis based caching mechanism then. But until now an in-memory routing service might be enough. What is your opinion? Do you see any objections against it?

CLAUDE

It's a clean simplification and for your scale it's entirely sound.

The real concern worth considering:

The only genuine concern is **multiple instances**. If you ever run more than one replica of the Custom Router for redundancy or load balancing, each instance has its own in-memory table. An invalidation event needs to reach all of them. With Redis as a shared cache this is automatic. With pure in-memory you need to either broadcast the invalidation webhook to all instances, or accept a TTL-based eventual consistency window across replicas.

The ones that don't really apply:

Memory size is a non-issue. Even at 10,000 customers with 20 pages each, the routing table is a few hundred kilobytes at most.

Process restart means a cold table, but your reconciliation-on-startup pattern already handles this.

Data loss on crash is irrelevant since Postgres is the source of truth.

My take:

Start with pure in-memory, no Redis. It reduces your infrastructure by one component, eliminates a network hop on every request, and makes the service simpler. If you later need multiple replicas, you can introduce Redis at that point.

This is a good example of not solving problems you don't have yet.

YOU

Ok. I will move forward with Go. However, I've never worked with Go before, I even don't know the language. But as this service will really be a few hundreds of lines of code, I'd like to see how would you implement it. Of course with verbose explanations because of my lack of Go knowledge.

CLAUDE

[A complete Go implementation was provided as `router.go` (~350 lines with detailed comments), along with `go.mod` and a `Dockerfile`.

Key components of the implementation:

RoutingTable struct — wraps a `map[string]RouteEntry` with a `sync.RWMutex`. Multiple goroutines can read simultaneously; writes get exclusive access.

RouteEntry struct — represents one domain's config: either `{"type": "site", "routes": {...}}` or `{"type": "redirect", "target": "..."}.`

reconcile() — runs a denormalizing SQL query against Postgres, builds the full routing map, and atomically replaces the in-memory table via `ReplaceAll()`.

reloadDomain() — re-fetches a single domain from Postgres. Called on invalidation events.

newProxyHandler() — the main request handler. Extracts `Host` header, looks up the routing table, rewrites the URL path, and proxies to WordPress using `httputil.ReverseProxy`.

newAdminHandler() — internal HTTP API on a separate port (`:8081`). Accepts `POST /invalidate` with a JSON body listing domains to reload, or triggering a full reconcile.

Two HTTP servers run concurrently via goroutines — `:8080` for public traffic, `:8081` for internal admin/webhook calls.

The `Dockerfile` uses a two-stage build: Go compiler image to build a static binary, then a `scratch` base image for the final container (~10MB total).]